

(✓) [Directed](#) ⓘ(✓) [Undirected](#) ⓘ(✓) [Heterogeneous nodes](#) ⓘ(✓) [Heterogeneous relationships](#) ⓘ(✓) [Weighted relationships](#) ⓘ

§ Introduction

Given a graph, a bridge is a relationship whose removal increases the number of connected components in the graph. Equivalently, a relationship can only be a bridge if and only if it is not contained in any cycle. The Neo4j GDS Library provides an efficient linear time sequential algorithm to compute all bridges in a graph.

For more information on this algorithm, see:

- [Bridge \(graph theory\)](#) ↗

Syntax

This section covers the syntax used to execute the Bridges algorithm in each of its execution modes. We are describing the named graph variant of the syntax. To learn more about general syntax variants, see [Syntax overview](#).

Bridges syntax per mode

[Stream mode](#)

AI search

Run Bridges in stream mode on a named graph.

CYPHER 

```
CALL gds.bridges.stream(  
  graphName: String,  
  configuration: Map  
)  
YIELD  
  from: Integer,  
  to: Integer  
  remainingSizes: List of Long
```

Table 1. Parameters

Name	Type	Default	Optional	Description
graphName	String	n/a	no	The name of a graph stored in the catalog.
configuration	Map	{}	yes	Configuration for algorithm-specifics and/or graph filtering.

Table 2. Configuration

Name	Type	Default	Optional	Description
<u>nodeLabels</u>	List of String	['*']	yes	Filter the named graph using the given node labels. Nodes with any of the given labels will be included.
<u>relationshipTypes</u>	List of String	['*']	yes	Filter the named graph using the given relationship types. Relationships with any of the given types will be included.
<u>concurrency</u>	Integer	1	yes	The algorithm is single-threaded and changing the concurrency parameter has no effect on the runtime.

Name	Type	Default	Optional	Description
<u>jobId</u>	String	Generated internally	yes	An ID that can be provided to more easily track the algorithm's progress.
<u>logProgress</u>	Boolean	true	yes	If disabled the progress percentage will not be logged.

Table 3. Results

Name	Type	Description
from	Integer	Start node ID.
to	Integer	End node ID.
splitComponents	List of Long	A list with the resulting sizes of the components after removing the relationship.

Examples

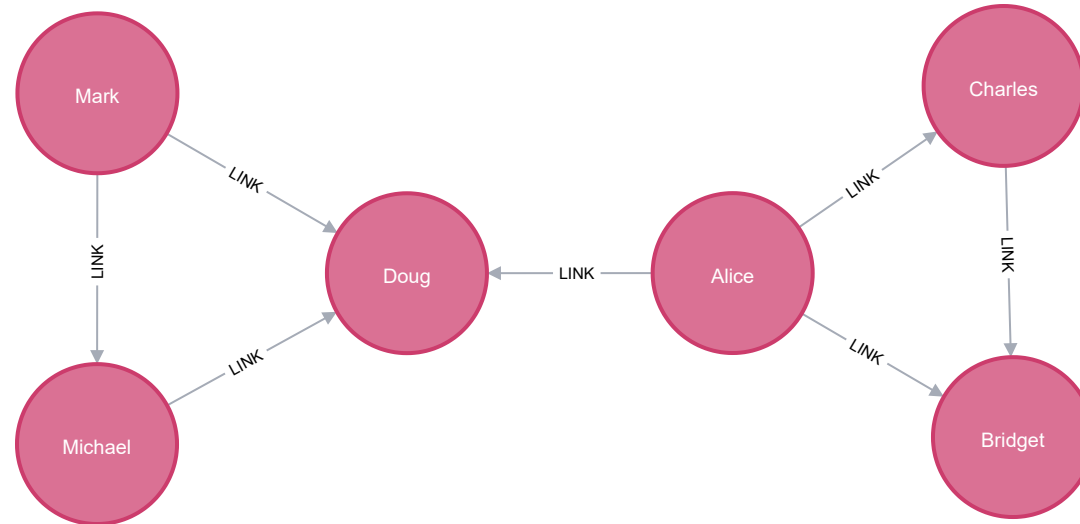
NOTE

All the examples below should be run in an empty database.

The examples use Cypher projections as the norm. Native projections will be deprecated in a future release.

In this section we will show examples of running the Bridges algorithm on a concrete graph. The intention is to illustrate what the results look like and to provide a guide in how to make use of the algorithm in a real setting. We will do this on a small social network

graph of a handful nodes connected in a particular pattern. The example graph looks like this:



The following Cypher statement will create the example graph in the Neo4j database:

CYPHER

CREATE

```
(nAlice:User {name: 'Alice'}),  
(nBridget:User {name: 'Bridget'}),  
(nCharles:User {name: 'Charles'}),  
(nDoug:User {name: 'Doug'}),  
(nMark:User {name: 'Mark'}),  
(nMichael:User {name: 'Michael'}),
```

```
(nAlice)-[:LINK]->(nBridget),  
(nAlice)-[:LINK]->(nCharles),  
(nCharles)-[:LINK]->(nBridget),
```

```
(nAlice)-[:LINK]->(nDoug),
```

```
(nMark)-[:LINK]->(nDoug),
```

```
(nMark)-[:LINK]->(nMichael),  
(nMichael)-[:LINK]->(nDoug);
```

This graph has two clusters of *Users*, that are closely connected. Between those clusters there is one single edge.

The following statement will project a graph using a Cypher projection and store it in the graph catalog under the name 'myGraph'.

CYPHER 

```
MATCH (source:User)-[r:LINK]->(target:User)  
RETURN gds.graph.project(  
  'myGraph',  
  source,  
  target,  
  {},  
  { undirectedRelationshipTypes: ['*'] }  
)
```

Memory Estimation

First off, we will estimate the cost of running the algorithm using the `estimate` procedure. This can be done with any execution mode. We will use the `stream` mode in this example. Estimating the algorithm is useful to understand the memory impact that running the algorithm on your graph will have. When you later actually run the algorithm in one of the execution modes the system will perform an estimation. If the estimation shows that there is a very high probability of the execution going over its memory limitations, the execution is prohibited. To read more about this, see [Automatic estimation and execution blocking](#).

For more details on `estimate` in general, see [Memory Estimation](#).

The following will estimate the memory requirements for running the algorithm:

CYPHER 

```
CALL gds.bridges.stream.estimate('myGraph', {})  
YIELD nodeCount, relationshipCount, bytesMin, bytesMax, requiredMemory
```

Table 4. Results

nodeCount	relationshipCount	bytesMin	bytesMax	requiredMemory
6	14	1152	1152	"1152 Bytes"

Stream

In the `stream` execution mode, the algorithm returns the bridge for each relationship. This allows us to inspect the results directly or post-process them in Cypher without any side effects.

For more details on the `stream` mode in general, see [Stream](#).

The following will run the algorithm in **stream** mode:

CYPHER 

```
CALL gds.bridges.stream('myGraph')  
YIELD from, to, remainingSizes  
RETURN gds.util.asNode(from).name AS fromName, gds.util.asNode(to).name AS toName, remainingSizes  
ORDER BY fromName ASC, toName ASC
```

Table 5. Results

fromName	toName	remainingSizes
"Alice"	"Doug"	[3, 3]

[Prev](#)[< Betweenness Centrality](#)[Next](#)[CELf >](#)

Contents

[Introduction](#)[Syntax](#)[Examples](#)[Memory Estimation](#)[Stream](#)

Graph Data Science courses on GraphAcademy

Follow our Graph Data Analytics learning path to learn how to apply graph thinking to your machine learning pipelines.

[Enroll now](#)

LEARN

-  [Sandbox](#)
-  [Neo4j Community Site](#)
-  [Neo4j Developer Blog](#)
-  [Neo4j Videos](#)
-  [GraphAcademy](#)
-  [Neo4j Labs](#)

SOCIAL

-  [Twitter](#)
-  [Meetups](#)
-  [Github](#)
-  [Stack Overflow](#)

[Want to Speak?](#)

CONTACT US →

US: 1-855-636-4532
Sweden +46 171 480 113
UK: +44 20 3868 3223
France: +33 (0) 1 88 46 13 20

© 2026 Neo4j, Inc.

[Terms](#) | [Privacy](#) | [Sitemap](#)

Neo4j[®], Neo Technology[®], Cypher[®], Neo4j[®] Bloom[™] and Neo4j[®] Aura[™] are registered trademarks of Neo4j, Inc. All other marks are owned by their

respective companies.

